Embedded Xinu
Raspberry Pi 5 (BCM2712)
User's Manual

Xinu-rpi5 Project

June 2026

Contents

1 Introduction

This document is the user's manual for the Embedded Xinu kernel (`xinu-rpi5`) that runs bare-metal on the Raspberry Pi 5 (BCM2712 / Cortex-A76). The system is a single-image AArch64 kernel with the MMU enabled, and provides:

- HDMI framebuffer and a window system (desktop)
- Wired Ethernet over RP1 (TCP/IP and an HTTP gateway)
- Full WiFi via the on-board CYW43455 (scan / WPA2 / DHCP / ping)
- USB mouse and keyboard via the RP1 xHCI
- microSD read/write (FAT32) mounted at `/microsd`
- A `kexec` selector that switches to another kernel in RAM
- Four OS variants derived from a single source tree

Important prerequisite (boot media). The board **boots from a USB stick**; the on-board microSD slot is normally empty. The firmware loads `kernel_2712.img` from the USB stick's FAT partition (`bootfs`). The microSD slot is used as a **data** area handled by this kernel's SD driver (mounted at `/microsd`).

2 System Overview

2.1 Key Specifications

Item	Value
SoC	BCM2712 (Cortex-A76, AArch64)
Kernel image	`kernel_2712.img` (entry `0x80000`)
MMU	Enabled (identity map, caches off)
Debug UART	`0x107D001000` (3-pin JST, 115200 8N1)
HDMI	Default 1920×1080×32 (OS3 uses 1280×720)
Wired IP (static)	`192.168.3.101`
microSD controller	sdhci-brcmstb sdio1 (`0x10_00FFF000`)
USB host	RP1 DWC3/xHCI (over PCIe)

2.2 Console

Output appears on both the HDMI text console and the debug UART (`uart_putc` is mirrored to the screen). When the desktop starts, the window system takes over the screen and the text console is hidden. Remote operation over the network (Section 8) is the most reliable way to drive the system.

3 Flashing and Booting the Kernel

3.1 Build

Run `make` in the `compile/` directory.

```
cd compile
make pi5         # Full OS          -> kernel_2712.img
make pi5-osmin   # OS1 (minimal)    -> kernel_min.img
make pi5-os2     # OS2 (no WM)       -> kernel_os2.img
make pi5-os3     # OS3 (lower res)  -> kernel_os3.img
```

3.2 Writing to the USB Stick (Mac)

Insert the USB stick (FAT partition = `bootfs`) into the Mac, copy the image, and eject. Always verify the mount point.

```
diskutil mount /dev/disk4s1
cp kernel_2712.img /Volumes/bootfs/kernel_2712.img
sync
diskutil unmount /dev/disk4s1
```

After writing, put the USB stick back into the Pi 5 and power it on. The boot log appears on the UART/HDMI after the firmware initialises (about ten-plus seconds).

4 The Four OS Variants and Switching with kexec

Four kernels are produced from the same `main.c` by changing only the compile flags. Any of them can be switched at runtime via `kexec` (Section 6).

Name	Image	Characteristics
Full OS	`kernel_2712.img`	Window system + networking (all features)
OS1	`OS1.IMG`	Shell only. No networking, no windows
OS2	`OS2.IMG`	All features but no window system (text console)
OS3	`OS3.IMG`	All features, HDMI lowered one rank to 1280×720

- **OS1 (minimal)**: no networking and no windows. Shows an `xinu-min$` prompt on the HDMI text console; driven by the USB keyboard.
- **OS2 (no WM)**: keeps all features (networking, etc.) but runs the shell on a full-screen text console instead of the window manager (prompt `xinu-os2$`). Remote operation over the network also works.
- **OS3 (lower resolution)**: same as the full OS but with HDMI at 1280×720.

Readability. The text-console font is the 8×8 glyph scaled 3× (active on OS1 and OS2).

5 microSD Storage

The on-board microSD slot is mounted at `/microsd` (the directory exists even with no card inserted). Reading a FAT32 card and persistently writing small files are both supported.

5.1 Listing and Reading

```
ls /microsd                 # list files on the card
cat /microsd/CONFIG.TXT     # print a file's contents
```

5.2 Writing (persistent)

A small file (up to one cluster, 32 KB) can be written persistently to the card over HTTP. The body is the content; the end of the URL is the file name (8.3 form).

```
curl -X POST --data-binary "hello from xinu" \
     http://192.168.3.101/microsd/write/TEST.TXT
# Survives a reboot: ls /microsd / cat /microsd/TEST.TXT
```

5.3 Diagnostics

A diagnostic command that shows the SD controller and card init state.

```
sdtest        # shows controller + CMD0/CMD8/ACMD41 responses
```

6 kexec Kernel Selector

`kexec` loads a kernel image from the microSD into RAM and jumps to it without a power cycle (no re-flash). Because it is a RAM-only operation, even a bad image is recovered by power-cycling back into the USB full OS; there is no brick risk.

```
kexec /microsd/OS1.IMG      # boot OS1 (minimal)
kexec /microsd/OS2.IMG      # boot OS2 (no WM)
kexec /microsd/OS3.IMG      # boot OS3 (lower resolution)
kexec /microsd/KERNEL~1.IMG # the full OS stored on the card
```

Notes.

- Networking drops after `kexec` (PCIe is re-initialised from a non-firmware state). Check the result on the HDMI screen.
- Only **bare-metal Xinu kernels** can be booted. A Linux kernel (e.g. `KERNEL8.IMG`, about 9.6 MB) needs a different boot protocol and would hang under the simple chainloader. This implementation refuses any image $\geq$ 4 MB as Linux (both Xinu and Linux are arm64 Images sharing the same magic, so size is used to tell them apart).

7 Shell Commands

Run from the on-screen shell (USB keyboard) or over the network (Section 8) via `/run?cmd=...`. The main commands:

Command	Description
`help`	list commands
`ls [path]`	list a directory
`cat <file>`	print a file's contents
`cd / pwd`	change / print the current directory
`kexec /microsd/<k.img>`	boot another kernel (Section 6)
`cc <file> / make <file>`	compile and run a C/AIPL program in place
`mem`	heap / BSS status
`peek <hex>`	read a 32-bit MMIO word
`uptime`	generic timer value
`ps`	core / EL status
`wifi on/off/status/scan`	connect / disconnect / state / scan (Section 9)
`wifi adhoc <ssid> [ch] [n]`	join a MANET ad-hoc (mesh) cell (Section 9)
`wifi aodv <a.b.c.d>`	discover a multi-hop mesh route (Section 9)
`sdtest`	SD controller diagnostics
`reboot`	reboot (PM watchdog)
`clear`	clear the screen

8 Remote Operation over the Network

The Full OS, OS2, and OS3 listen for HTTP on the wired IP `192.168.3.101`. Even where serial input is awkward, the system can be driven with `curl` from a Mac or similar.

Endpoint	Description
/run?cmd=<shell>	run any shell command and return its output
/fs, /fs/ls/<d>, /fs/cat/<f>	browse the VFS tree
/fs/write/<f> (POST)	write a file to the VFS (tmpfs)
/microsd/write/<NAME> (POST)	persistent write to microSD (Section 5)
/usb/...	USB enumeration and diagnostics (Section 10)
/api/actors, /send?to=&m=	actor / AIPL gateway

```
curl 'http://192.168.3.101/run?cmd=ls%20/microsd'
curl 'http://192.168.3.101/run?cmd=cat%20/microsd/CONFIG.TXT'
curl 'http://192.168.3.101/run?cmd=wifi%20status'
```

Note: encode spaces in the URL as %20 or +. The output buffer is bounded, so long results are split.

9 WiFi

Full WiFi via the on-board CYW43455 (scan / WPA2 / DHCP / ping) is supported. The system **does not auto-connect at boot**; it connects only when wifi on is run.

9.1 Connecting to an access point

```
wifi on <ssid> <pass>   # bring up the firmware (~10 s), join WPA2, run DHCP
wifi on                 # reuse the last creds, or the build-time wifi.conf
wifi scan               # bring up the radio and list nearby APs
wifi status             # connection state, SSID, IP
wifi off                # radio down
wifi-invest             # diagnostics when the link won't come up
```

A successful wifi on ends with wifi: CONNECTED IP=.... A WiFi signal icon is drawn at the bottom-right of the screen, with the SSID and local IP beneath it. Credentials are embedded into the kernel at build time from a .gitignored wifi.conf (never commit the password).

9.2 Mesh networking (MANET ad-hoc)

Several Pi 5 (and Pi 4 / Pi 3) Xinu nodes can form a peer-to-peer mesh with **no access point**, using 802.11 IBSS (ad-hoc) mode plus on-demand AODV multi-hop routing.

```
wifi adhoc <cell-ssid> [ch] [node]   # join an ad-hoc cell as 10.0.0.<node>
wifi aodv  <a.b.c.d>                 # discover a multi-hop route on demand
```

- wifi adhoc mesh1 6 1 brings the radio up in IBSS mode on the named cell (BSSID 02:4d:41:4e:45:54 = "MANET"), channel 6 (default), and takes the static IP 10.0.0.1 (the [node] number, default 1); the AODV relay then runs automatically.
- All nodes use the **same cell SSID and channel, with a distinct [node] number** (10.0.0.<node>/24). Nodes in radio range reach each other directly at 10.0.0.x.
- For a destination beyond direct range, wifi aodv 10.0.0.3 broadcasts an RREQ (UDP 654); intermediate nodes relay it and the destination replies with an RREP, installing a multi-hop route (e.g. AODV route: 10.0.0.3 via 10.0.0.2, 2 hop). Run wifi adhoc first so the node has an IP.

Ad-hoc/AODV is independent of the infrastructure wifi on mode and is not restored after reboot — re-run wifi adhoc on each node.

10 USB (Mouse and Keyboard)

The USB-A ports connect to two DWC3/xHCI controllers inside the RP1. To avoid faulting the boot, enumeration runs automatically a few seconds after boot (or can be driven manually via the `/usb/...` endpoints). The boot mouse moves the cursor and the keyboard feeds the shell.

11 Appendix

11.1 Build Targets

Target	Output
`make pi5`	`kernel_2712.img` (Full OS)
`make pi5-osmin`	`kernel_min.img` (OS1)
`make pi5-os2`	`kernel_os2.img` (OS2)
`make pi5-os3`	`kernel_os3.img` (OS3)

11.2 Troubleshooting

- **HTTP not responding**: right after boot the network is not yet up. Wait ten-plus seconds and retry. Networking drops after `kexec`; power-cycle to return to the USB full OS.
- **`kexec` says "too large"**: an image $\geq$ 4 MB (e.g. a Linux kernel) cannot be chainloaded. Specify a bare-metal Xinu image.
- **`/microsd` is empty**: check that a microSD is in the slot. Use `sdtest` to inspect the init state.
- **No display**: if the HDMI framebuffer mailbox init fails, the console is UART-only.

11.3 Safety Notes

- Never commit the WiFi password to the source repository (it is embedded at build time from a `.gitignore`d file).
- `kexec` is a RAM-only operation; even an invalid image is recovered by power-cycling (no risk of flash corruption).